Document Databases

1. Overview

Document databases store data as self-contained **documents** instead of rows in tables.

A document usually contains structured fields in formats such as:

- JSON
- BSON
- YAML
- XML (some systems)

Example:

```JSON
{
  "user_id": 101,
  "name": "Samarth",
  "city": "Surat",
  "skills": ["React", "Node"]
}
```

They are a major category of **NoSQL databases**.

2. Simple Interpretation

A document database can be viewed as an advanced key-value store:

```
None
key      → document

user:101 → {...JSON...}
```

But unlike simple key-value stores, document DBs can query inside document fields.

3. Popular Examples

- MongoDB
- CouchDB
- Amazon DocumentDB

4. Core Structure

Instead of tables:

```
None
Database

 └── Collection

      └── Documents
```

Equivalent idea:

SQL	Document DB
Database	Database
Table	Collection
Row	Document

5. Example Collection

Users Collection

JSON

```
{
  "_id": 101,
  "name": "Samarth",
  "email": "sam@email.com",
  "address": {
    "city": "Surat",
    "state": "Gujarat"
  }
}
```

6. Why Document DBs Are Popular

Flexible schema
Natural JSON storage
Fast development
Easy nested data storage
Good horizontal scaling
Good for evolving products

7. Flexible Schema

Different documents can have different fields.

```JSON
{ "name":"A", "age":21 }

{ "name":"B", "skills":["C++"] }
```

Unlike rigid SQL schemas.

Useful when product changes often.

8. Embedded/Nested Data

Store related data together.

```json
{
  "order_id": 5001,
  "user_id": 101,
  "items": [
    {"name":"Laptop","qty":1},
    {"name":"Mouse","qty":2}
  ]
}
```

Good for read-heavy use cases.

9. Querying Inside Documents

Example:

```
None
Find users where city = Surat

Find orders containing Laptop
```

More powerful than plain key-value stores.

10. Best Use Cases

Use document DB for:

- User profiles
- Product catalogs
- CMS systems
- Blogging platforms
- Dynamic forms
- Inventory with changing attributes
- Mobile app backends

11. Example: Product Catalog

Different products need different fields.

```JSON
{
  "type":"phone",
  "ram":"8GB",
  "camera":"50MP"
}
```

```JSON
{
  "type":"shoe",
  "size":9,
  "material":"leather"
}
```

Harder in fixed SQL schema.

12. Document DB vs SQL

Feature	SQL	Document DB
Schema	Fixed	Flexible
Joins	Strong	Limited / avoid
Nested data	Less natural	Excellent
Transactions	Strong	Varies
Horizontal scaling	Moderate	Better often

13. Document DB vs Key-Value

Feature	Key-Value	Document DB
Store values	Yes	Yes
Query inside value	Usually limited	Strong
Secondary indexes	Limited	Common
Structured docs	Basic	Native

14. MongoDB Example

MongoDB stores BSON documents and supports:

- Indexes
- Replication
- Sharding
- Aggregation pipelines
- Transactions (modern versions)

15. Scaling

Common methods:

- Replication for HA
- Sharding for scale
- Read replicas

Good for growing apps.

16. Trade-offs

Complex joins weaker than SQL
Data duplication common
Poor schema discipline can create chaos
Multi-document consistency may need care

17. Common Design Pattern

Use SQL + Document DB together:

```
None
SQL → Payments / Orders

MongoDB → Profiles / Catalog / Content
```

18. Interview Answer Example

Q: Why MongoDB here?

> I'd choose MongoDB because the data model changes frequently, nested JSON fits naturally, and we need horizontal scalability with rapid development speed.

19. Short Revision Notes

```
None
Document DB:

- Stores JSON/BSON docs

- Flexible schema

- Nested objects

- Good for dynamic apps


Examples:

MongoDB, CouchDB
```

20. One-Line Summary

Document databases store semi-structured records as JSON-like documents, making them ideal for flexible schemas, nested data, and rapidly evolving applications.